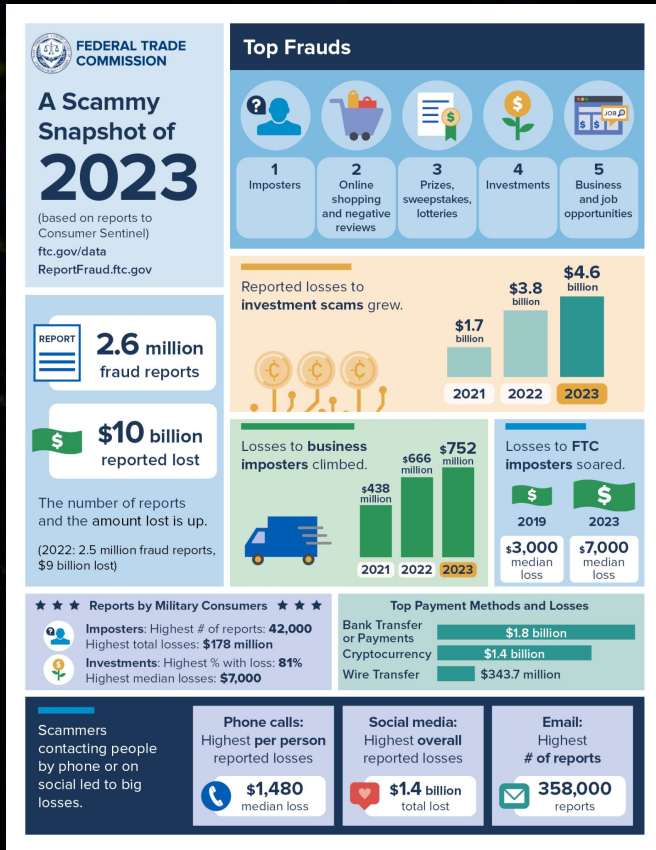


CS 471 Final Project

Shane Dyrdaahl, Alastair Raymond,
Riley Saliba, Dax Taraleskof

Motivation



Prior Research

International Journal of Academic Information Systems Research (IJAISR)

ISSN: 2643-9026

Vol. 6 Issue 3, March - 2022, Pages:30-39

Fraudulent Financial Transactions Detection Using Machine Learning

Mosa M. M. Megdad, Bassem S. Abu-Nasser and Samy S. Abu-Naser

Goals

- Flag potentially fraudulent transactions
- Focus on lower inference time
- Avoid missing false negatives (high recall)





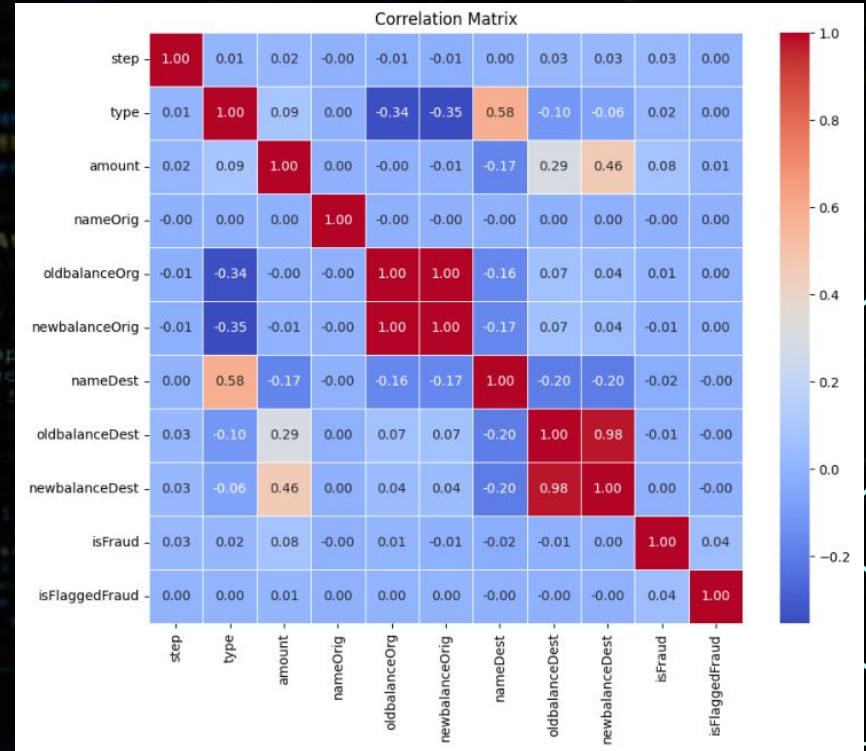
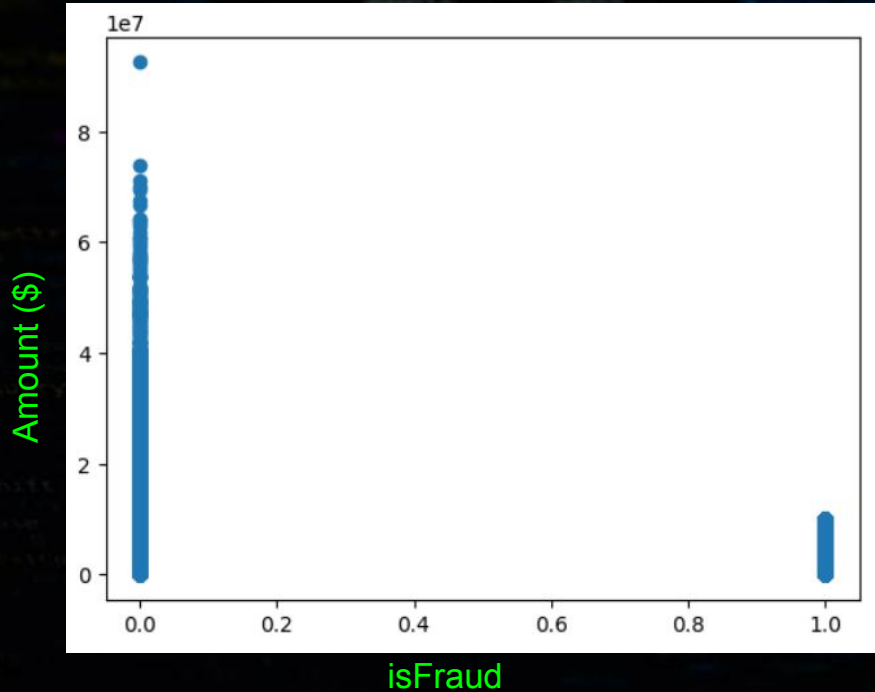
VARDHAN SIRAMDASU • UPDATED 3 YEARS AGO

Fraudulent Transactions Prediction

predicting fraudulent transactions

type	nameDest
amount	oldbalanceDest
nameOrig	newbalanceDest
oldbalanceOrig	isFraud
newbalanceOrig	

Dataset Analysis



6,354,407 non fraud (99.87%)
8,213 fraud (0.13%)

Data Pre-Processing

Label Encoding

type

nameOrig

nameDest

 [Install](#) [User Guide](#) [API](#) [Examples](#) [Community](#) [More](#)

train_test_split

`sklearn.model_selection.train_test_split(*arrays, test_size=None, train_size=None, random_state=None, shuffle=True, stratify=None)` [\[source\]](#)

 [Getting Started](#) [User Guide](#) [API reference](#) [Examples](#) [Release history](#) [More](#)

SMOTE

`class imblearn.over_sampling.SMOTE(*, sampling_strategy='auto', random_state=None, k_neighbors=5, n_jobs=None)` [\[source\]](#)

Algorithm 1

Rules-Based Approach

```
# Function to evaluate the rules-based model.
def evaluate(threshold, data, actual):
    # Identify potential fraudulent transactions based on the threshold.
    potential_fraud = merged_data.loc[(merged_data['amount'] < threshold)]
    actual_fraud = potential_fraud.loc[(potential_fraud['isFraud'] == 1)]
    not_fraud = potential_fraud.loc[(potential_fraud['isFraud'] == 0)]

    # Calculate recall, precision, and F1-score.
    true_positive_count = len(actual_fraud)
    total_fraud_count = len(merged_data.loc[(merged_data['isFraud'] == 1)])
    false_negative_count = total_fraud_count - len(actual_fraud)
    false_positive_count = len(not_fraud)

    # Calculate recall metric
    recall = (true_positive_count / (true_positive_count + false_negative_count))
    print("Recall: " + str(recall))
```

```
evaluate(10000000, evaluation_X, evaluation_y)
```

Algorithm 2 (ML Approach)

Decision Tree

```
# Define hyperparameters for Decision Tree tuning.
param_grid_dt = {
    'max_depth': [4],
    'min_samples_split': [2]
}

# Train Decision Tree with GridSearchCV.
# Tune the hyperparams and then fit the training data on it
decision_tree = DecisionTreeClassifier(random_state=130)
grid_search_dt = GridSearchCV(estimator=decision_tree, param_grid=param_grid_dt, cv=5, scoring='recall', n_jobs=-1,
                              verbose=1)
grid_search_dt.fit(training_X, training_y)

# Evaluate on evaluation dataset and tune hyperparameters based on result
dt_evaluate(evaluation_X, evaluation_y, grid_search_dt)
```

Algorithm 3 (ML Approach)

Random Forest

```
# Define hyperparameters for Random Forest tuning.
# using best params from FirstTrainingRF.png
param_grid_rf = {
    'n_estimators': [25], # Number of trees in the forest.
    'max_depth': [2], # Maximum tree depth.
    'min_samples_leaf': [1] # Minimum samples at leaf nodes.
}

# Train Random Forest with GridSearchCV.
random_forest = RandomForestClassifier(class_weight='balanced',
random_state=130)
grid_search_rf = GridSearchCV(estimator=random_forest,
param_grid=param_grid_rf, scoring='recall', n_jobs=-1, verbose=1)
# random forest wants a 1d array, hence the ravel()
grid_search_rf.fit(training_X, training_y.values.ravel())

# Evaluate the Random Forest model on evaluation and testing sets.
rf_evaluate(evaluation_X, evaluation_y, grid_search_rf)
```

Rules Based Experiments

Threshold = Mode

Median = 172383.67472022248

Maximum = 71172480.42

Mode = 10000000.0

```
C:\Users\pinoy\AppData\Local\Temp\ipykernel_13288\2
mode = float(training_X[['amount']].mode().iloc[0])
```

```
evaluate(10000000, evaluation_X, evaluation_y)
```

✓ 0.4s

Recall: 0.966543822506257

Precision: 0.49158243935633655

F1-Score: 0.6517075817638599

Threshold = Double the Mode

Median = 172383.67472022248

Maximum = 71172480.42

Mode = 10000000.0

```
C:\Users\pinoy\AppData\Local\Temp\ipykernel_13288\2
mode = float(training_X[['amount']].mode().iloc[0])
```

```
evaluate(20000000, evaluation_X, evaluation_y)
```

✓ 0.4s

Recall: 1.0

Precision: 0.49991614488786285

F1-Score: 0.6665921246220571

Rules Based Experiments

Threshold = Max

Median = 172383.67472022248

Maximum = 71172480.42

Mode = 10000000.0

[C:\Users\pinoy\AppData\Local\Temp\ipykernel_13288\27732158](#)

```
mode = float(training_X[['amount']].mode().iloc[0])
```

```
evaluate(71172480.42, evaluation_X, evaluation_y)
```

✓ 0.4s

Recall: 1.0

Precision: 0.49988000487849016

F1-Score: 0.6665599958031135

Threshold = Median

Median = 172383.67472022248

Maximum = 71172480.42

Mode = 10000000.0

[C:\Users\pinoy\AppData\Local\Temp\ipykernel_13288\277321585.py](#)

```
mode = float(training_X[['amount']].mode().iloc[0])
```

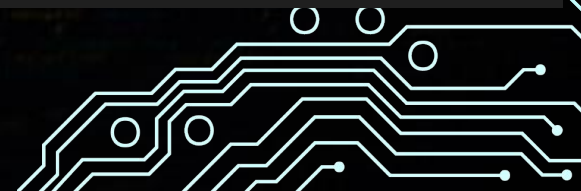
```
evaluate(172383.67472022248, evaluation_X, evaluation_y)
```

✓ 0.3s

Recall: 0.30721304443639913

Precision: 0.30718493896554744

F1-Score: 0.30719899105813464



DT Experiments

```
# hyperparams to tune
param_grid = {
#   'max_depth': [i for i in range(2, 5)],
|   'max_depth': [6],
#   'min_samples_leaf': [i for i in range(2, 5)]
|   'min_samples_leaf': [2]
}
```

[22] ✓ 0.0s

```
# tune the hyperparams and then fit the training data on it
decision_tree = DecisionTreeClassifier(random_state=130)
grid_search = GridSearchCV(estimator=decision_tree, param_grid=param_grid, cv=5, scoring='recall', n_jobs=-1, verbose=1)
grid_search.fit(training_X, training_y)
```

[23] ✓ 2m 44.8s

... Fitting 5 folds for each of 1 candidates, totalling 5 fits

GridSearchCV

```
GridSearchCV(cv=5, estimator=DecisionTreeClassifier(random_state=130),
             n_jobs=-1, param_grid={'max_depth': [6], 'min_samples_leaf': [2]},
             scoring='recall', verbose=1)
```

best_estimator_: DecisionTreeClassifier

```
DecisionTreeClassifier(max_depth=6, min_samples_leaf=2, random_state=130)
```

DecisionTreeClassifier

```
DecisionTreeClassifier(max_depth=6, min_samples_leaf=2, random_state=130)
```

DT Experiments

```
# perform final testing on testing dataset  
dt_evaluate(testing_X, testing_y, grid_search)
```

✓ 1.4s

best params: {'max_depth': 6, 'min_samples_leaf': 2}

best recall from gridsearch tuning: 99.63%

recall of best estimator from GridSearchCV on test set: 99.76%

	precision	recall	f1-score	support
isNotFraud	1.00	0.98	0.99	1270882
isFraud	0.06	1.00	0.12	1642
accuracy			0.98	1272524
macro avg	0.53	0.99	0.56	1272524
weighted avg	1.00	0.98	0.99	1272524

Confusion matrix:

```
[[1247154  23728]  
 [      4  1638]]
```

Model Comparison

Rules-Based

Recall: 0.9673355371185738
Precision: 0.49207580577396315
F1-Score: 0.6523211104252215

Decision Tree

```
Confusion matrix (DT):  
[[ 973505  43175]  
 [ 2664 1014059]]  
Best params: {'max_depth': 4, 'min_samples_split': 4}  
Best recall from GridSearchCV tuning: 99.73%  
recall of best estimator from GridSearchCV on test set for DT: 99.38%  
precision    recall  f1-score   support  
  
isNotFraud    1.00    0.96    0.98   1270901  
isFraud        0.03    0.99    0.06    1623  
  
accuracy              0.96   1272524  
macro avg    0.51    0.98    0.52   1272524  
weighted avg    1.00    0.96    0.98   1272524
```

Random Forest

```
best params: {'max_depth': 2, 'min_samples_leaf': 1, 'n_estimators': 25}  
best recall from gridsearch tuning: 93.13%  
recall of best estimator from GridSearchCV on test set for RF: 91.62%  
precision    recall  f1-score   support  
  
isNotFraud    1.00    0.93    0.96   1270866  
isFraud        0.02    0.92    0.03    1658  
  
accuracy              0.93   1272524  
macro avg    0.51    0.92    0.50   1272524  
weighted avg    1.00    0.93    0.96   1272524  
  
Confusion matrix (RF):  
[[1182378  88488]  
 [   139  1519]]
```

Performance Summary (Recall)		
Rules-based	Decision Tree	Random Forest
0.96	0.96	0.93

Overview of Hyperparameter Tuning

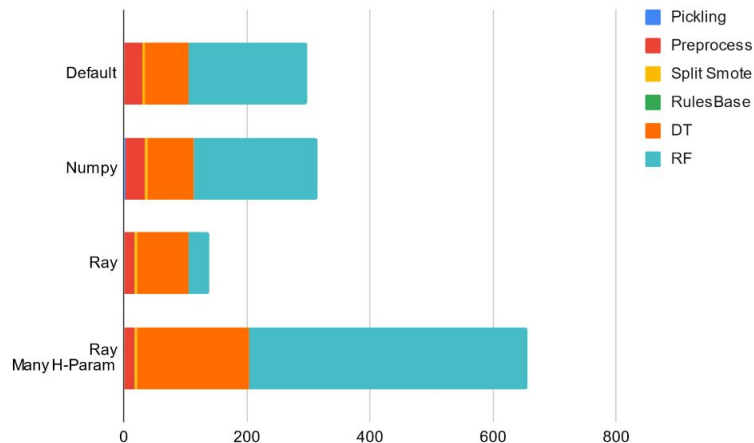
- **Additional Libraries Utilized**

- **Ray Tune:** Efficient distributed hyperparameter tuning.
- **Dask:** Preprocessing large datasets efficiently.
- **NumPy:** Improved computational efficiency in preprocessing.

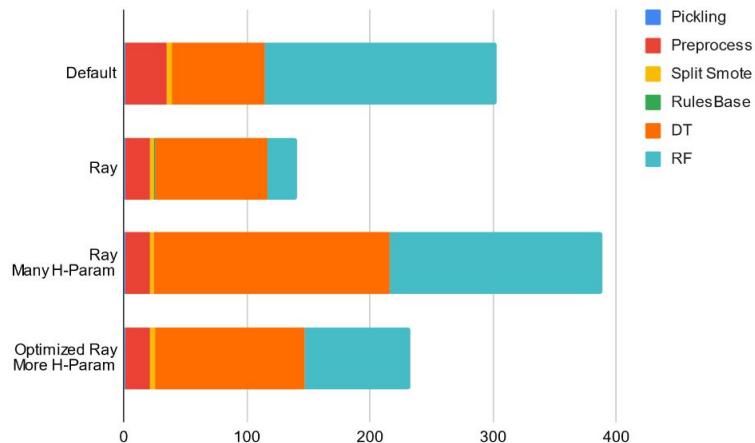
- **EC2 Instance:**

- 128 vCPU, 516 GiB RAM

Locally Hosted



AWS EC2



Parameter Tuning Thoughts

-
-
- Balancing CPU, memory allocation, and trial concurrency was crucial to fully utilize AWS resources.
- Failure Proof EC2 Deployment



Questions

```
(venv) [ec2-user@ip-172-31-12-67 ~]$ python main_ray.py
Starting 2
```

[illegible]